



**WYDZIAŁ
ELEKTROTECHNIKI
I INFORMATYKI**
POLITECHNIKI RZESZOWSKIEJ

Jakub Skrzynecki

Zagrożenia bezpieczeństwa systemów IT wynikające ze złośliwego
użycia modeli AI oraz kradzież danych przy pomocy metod Prompt
Injection

PROJEKT

Rzeszów, 2026

Wykaz skrótów

API	Application Programming Interface
ASR	Attack Success Rate
BCC	Blind Carbon Copy
CFO	Chief Financial Officer
CTA	Call to Action
CVE	Common Vulnerabilities and Exposures
CVSS	Common Vulnerability Scoring System
DLP	Data Loss Prevention
DPO	Direct Preference Optimization
GCG	Greedy Coordinate Gradient
GenAI	Generative Artificial Intelligence
GPAI	General-Purpose AI
HMAC	Hash-based Message Authentication Code
ICS/OT	Industrial Control Systems / Operational Technology
LLM	Large Language Model
MCP	Model Context Protocol
NIST	National Institute of Standards and Technology
NVD	National Vulnerability Database
OWASP	Open Worldwide Application Security Project
PDF	Portable Document Format
PII	Personally Identifiable Information
RAG	Retrieval-Augmented Generation
RCE	Remote Code Execution
RMF	Risk Management Framework
SCADA	Supervisory Control and Data Acquisition
SQL	Structured Query Language
XSS	Cross-Site Scripting

1. Wprowadzenie

Modele sztucznej inteligencji oparte na architekturze Transformer, znane jako duże modele językowe (ang. *Large Language Models*, LLM), w ciągu ostatnich trzech lat zmieniły sposób, w jaki firmy i użytkownicy indywidualni korzystają z technologii. ChatGPT, Claude, Gemini i dziesiątki innych systemów trafiły do codziennego użytku, od pisania kodu po obsługę klienta. Jednocześnie pojawiły się nowe kategorie zagrożeń, których tradycyjne modele bezpieczeństwa IT nie przewidywały.

Rozwój modeli AI postępuje równolegle ze wzrostem skuteczności nowych wektorów ataku. Według raportu KnowBe4 z 2025 roku około 82,6% phishingu wykazuje cechy treści generowanych przez AI [13]. OWASP klasyfikuje Prompt Injection na pierwszej pozycji listy Top 10 zagrożeń dla aplikacji LLM zarówno w edycji 2023/24, jak i 2025 [1], potwierdzając realność zagrożenia w produkcyjnych systemach komercyjnych.

1.1. Cel i zakres pracy

Celem projektu jest systematyczna analiza zagrożeń bezpieczeństwa systemów IT wynikających ze złośliwego użycia dużych modeli językowych, ze szczególnym uwzględnieniem technik Prompt Injection i ich wykorzystania do kradzieży danych. Zakres pracy obejmuje:

- taksonomię ataków bezpośrednich i pośrednich, w tym nowe kategorie zagrożeń pojawiające się w ekosystemach agentowych (Tool Poisoning, ataki w protokole MCP, ataki wielomodalne i zautomatyzowane);
- analizę rzeczywistych incydentów bezpieczeństwa udokumentowanych w literaturze branżowej i rejestrach podatności (CVE-2025-53773, CamoLeak, atak na SCADA przez Claude z MCP, incydenty Slack AI, ServiceNow);
- przegląd metod obrony na poziomie indywidualnym, korporacyjnym oraz regulacyjnym (EU AI Act, NIST AI RMF, OWASP Top 10 dla LLM 2025);
- krytyczną ocenę skuteczności i ograniczeń poszczególnych mechanizmów ochronnych.

1.2. Metodyka i źródła

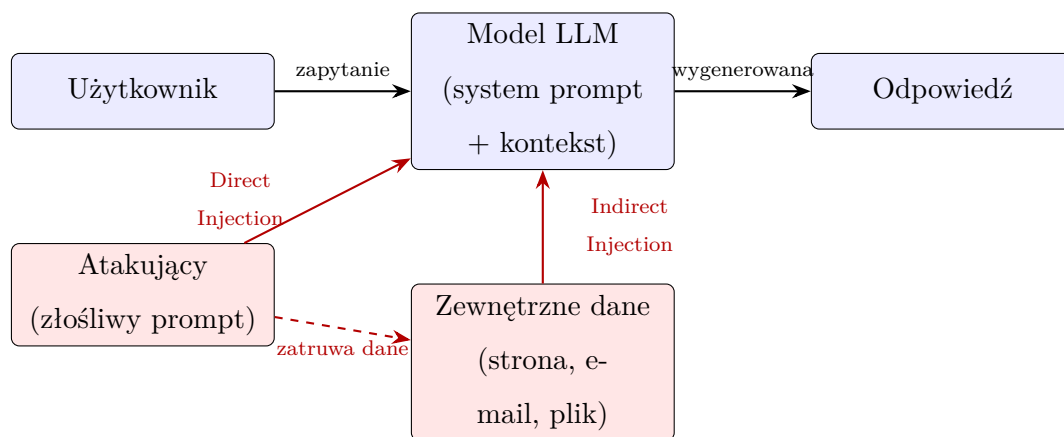
Opracowanie ma charakter przeglądu literaturowego z analityczną oceną materiału. Bazuje na recenzowanych publikacjach naukowych z lat 2023, 2024, 2025 oraz pierwszego kwartału 2026 (m.in. USENIX Security, MDPI *Information*, ScienceDirect), raportach standaryzacyjnych (OWASP, NIST), rejestrze NVD, dokumentacji dostawców (Microsoft, OpenAI, IBM, Datadog)

oraz analizach branżowych ujawniających rzeczywiste przypadki ataków. Łącznie wykorzystano 29 pozycji bibliograficznych. Twierdzenia ilościowe są podpierane bezpośrednim odwołaniem do źródła, a każdy z opisanych przypadków rzeczywistych jest powiązany z co najmniej jedną referencją.

2. Prompt Injection: definicja i taksonomia

2.1. Czym jest Prompt Injection

Prompt Injection to technika ataku polegająca na tym, że atakujący wstrzykuje do modelu językowego instrukcje, które zmieniają jego zachowanie w sposób niezamierzony przez twórcę systemu. Problem wynika z tego, że LLM-y przetwarzają instrukcje systemowe i dane użytkownika w tym samym kanale tekstowym, bez twardego rozgraniczenia między nimi [2]. Sytuacja przypomina pole formularza, w którym mieszają się komendy SQL i dane wejściowe użytkownika, z tą różnicą, że tutaj granica jest jeszcze bardziej rozmyta, ponieważ model operuje na języku naturalnym. Dwa podstawowe warianty ataku, czyli Direct Prompt Injection (wstrzyknięcie poprzez wejście użytkownika) oraz Indirect Prompt Injection (wstrzyknięcie poprzez zewnętrzne źródła danych), przedstawiono schematycznie na rys. 2.1.



Rys. 2.1. Schemat ataków Direct i Indirect Prompt Injection

Analogia do SQL Injection jest zresztą celowa i często pojawia się w literaturze. Różnica polega na tym, że SQL Injection dotyczy języka formalnego z jasno zdefiniowaną składnią, przez co da się zbudować parser, który wykryje większość prób ataku. W przypadku Prompt Injection mamy do czynienia z językiem naturalnym, co sprawia, że nie istnieje prosty filtr, który oddzieli złośliwe instrukcje od zwykłego tekstu [3].

OWASP klasyfikuje Prompt Injection jako główne zagrożenie aplikacji opartych na modelach językowych (LLM01:2025), uznając podatność systemów zarówno na manipulacje bezpośrednie, jak i pośrednie [1].

2.2. Direct Prompt Injection

Bezpośredni Prompt Injection to najprostsza forma ataku. Użytkownik wpisuje w polu czatu lub w zapytaniu API instrukcję, która ma nadpisać zachowanie modelu. Typowe przykłady to polecenia w stylu „zignoruj wszystkie wcześniejsze instrukcje” lub „udawaj, że jesteś administratorem systemu”. Studia przypadków pokazują, że nawet krótkie i niewyrafinowane prompty potrafią skutecznie nadpisywać zachowanie ChatGPT, co potwierdza analiza Wang i in. [4] dotycząca lekkich ataków na asystentów konwersacyjnych.

Badacze zidentyfikowali kilka powtarzalnych mechanizmów stosowanych w atakach bezpośrednich. Atak **DAN** (ang. *Do Anything Now*) narzuca modelowi przyjęcie nowej roli przez utworzenie odseparowanej tożsamości z usuniętymi ograniczeniami bezpieczeństwa (tzw. guardrails) [3]. Bardziej wyrafinowana technika **Deceptive Delight** wplata wrogie polecenia w kontekst pozytywnej narracji; atakujący prowadzi model do pominięcia filtrów przez stopniową zmianę intencji zapytania bazowego [3]. Atak **Fake Completion** manipuluje natomiast danymi początkowymi, podkładając sfalszowaną wcześniejszą wypowiedź modelu, aby zmusić go do kontynuowania instrukcji umieszczonej bezpośrednio po niej. Zestawienie najczęściej spotykanych technik Direct Prompt Injection wraz z ich mechanizmami i typowymi celami zawiera tab. 2.1.

Tabela 2.1. Porównanie wybranych technik Direct Prompt Injection

Technika	Mechanizm	Typowy cel
DAN (Do Anything Now)	Stworzenie fikcyjnej tożsamości AI bez ograniczeń	Obejście guardrailsów, generowanie zabronionych treści
Deceptive Delight	Wieloetapowe kamuflowanie złośliwych instrukcji	Stopniowe przejście zachowania modelu
Fake Completion	Wklejenie fałszywej odpowiedzi modelu do promptu	Kontynuacja złośliwego skryptu przez model
Kodowanie (Base64, Leet-speak)	Ukrywanie słów kluczowych przed filtrami	Ominięcie list zabronionych słów
ArtPrompt	Zastąpienie zabronionych słów sztuką ASCII	Ominięcie filtrów semantycznych

Istotnym wektorem ataku pozostaje kodowanie i modyfikacja wyrazów. Atakujący modyfikują zabronione wyrażenia, korzystając z Base64, notacji Leetspeak (np. !gn0r3 4ll 1nstruct10ns) lub niewidocznych znaków Unicode. Wariantem tej techniki jest **ArtPrompt**: zabronione wyrazy zostają zastąpione obrazami ASCII, które LLM samodzielnie rozszyfrowuje, omijając filtry semantyczne operujące na tekście jawnym [3].

Realne zagrożenie stwarzają ataki narastające stopniowo. Atak zaczyna się od niewinnych poleceń, na przykład prośby o powtórzenie komendy systemowej, a kończy wymuszeniem destrukcyjnych operacji lub wycieku danych.

2.3. Indirect Prompt Injection

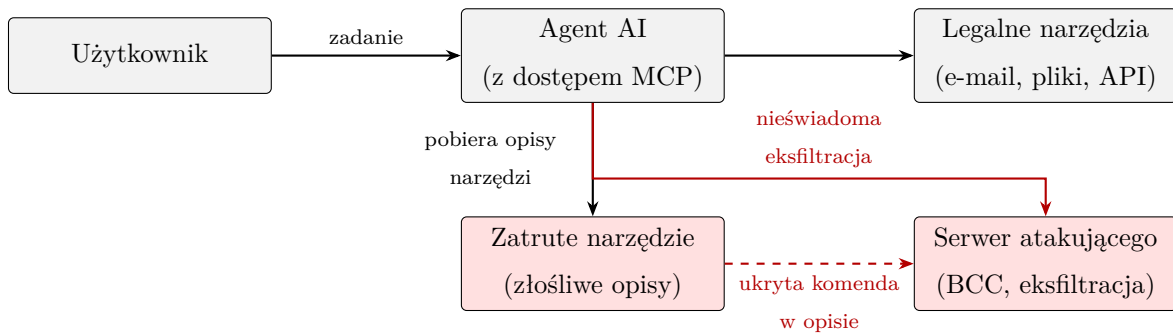
Pośredni Prompt Injection jest znacznie bardziej niebezpieczny, bo ofiara nie musi sama wpisywać złośliwej treści. Zamiast tego atakujący umieszcza instrukcje w źródłach danych, które model przetwarza: na stronach internetowych, w e-mailach, dokumentach, rekordach baz danych, a nawet zaproszeniach kalendarza [2].

Stored Prompt Injection to wariant, w którym złośliwy ładunek jest trwale umieszczany w pamięci długoterminowej modelu lub w bazie danych RAG (ang. *Retrieval-Augmented Generation*). Zaatakowany system niesie złośliwą instrukcję we wszystkich kolejnych sesjach, a jej działanie przypomina koncepcję wirusa działającego między sesjami. Mechanizm ten został udokumentowany jako „spAIware” w przypadku podatności ChatGPT [12].

Kaya i in. [5] analizują na dużą skalę podatności wtyczek czatbotów AI na stronach WWW i pokazują m.in. ryzyko manipulacji historią konwersacji oraz pośredniego Prompt Injection przez treści stron. Microsoft opublikował w 2025 r. szczegółowy opis metod obrony przed pośrednim Prompt Injection w swoich produktach [6].

2.4. Tool Poisoning i ataki agentowe

Wraz z rozwojem agentowych systemów AI, w których model nie tylko generuje tekst, ale wykonuje autonomiczne akcje, pojawiła się nowa kategoria zagrożeń. Standardy takie jak MCP (*Model Context Protocol*) definiują sposób komunikacji modeli AI z zewnętrznymi narzędziami i usługami. Sam protokół staje się jednak wektorem ataku w technikach określanych jako **Tool Poisoning** [29]. Schemat ataku, obejmujący zatrucie opisu narzędzia widocznego dla modelu i uruchomienie ukrytej instrukcji w ekosystemie MCP, prezentuje rys. 2.2.



Rys. 2.2. Mechanizm ataku Tool Poisoning przez zatrute narzędzia MCP

W ataku Tool Poisoning atakujący konfiguruje złośliwą wtyczkę lub narzędzie w ekosystemie MCP. Opisy narzędzi widoczne dla modelu, ale niewidoczne dla użytkownika, zawierają ukryte instrukcje, na przykład polecenie dołączania kopii ukrytej (ang. *Blind Carbon Copy*, BCC) do wszystkich wysyłanych e-maili na adres atakującego. Gdy agent czyta metadane narzędzi, przyswaja złośliwą instrukcję i od tej chwili wykonuje ją przy każdej operacji, udając legalne działanie [29].

Powiązana technika są **Rug Pull attacks**. Atakujący publikuje pozornie bezpieczne narzędzie, buduje zaufanie użytkowników, a następnie aktualizuje jego opisy o złośliwe instrukcje już po zdobyciu dostępu do danych ofiar. Asymetria tego ataku, polegająca na tym, że ofiary nie weryfikują opisów narzędzi przy każdym wywołaniu, czyni go trudnym do wykrycia.

Osobnym wektorem są ataki na systemy wieloagentowe: w modelach złożonych z wielu agentów i konektorów (w tym protokołów typu MCP) literatura wskazuje na eskalację zaufania i możliwość przenoszenia złośliwych instrukcji między rolami [29]. W produktowych platformach enterprise pojawiają się także formalnie zarejestrowane podatności, np. CVE-2025-12420 w ekosystemie ServiceNow AI (m.in. podszywanie się pod innego użytkownika przy szerokich uprawnieniach agentów) [23].

2.5. Ataki wielomodalne i zautomatyzowane

Modele multimodalne przyjmujące zarówno tekst, jak i obrazy otwierają nowe wektory ataku. **Mind Mapping Prompt Injection** polega na ukryciu szkodliwej instrukcji jako niedokończonego węzła mapy myśli w formie obrazu. Model, widząc niekompletną mapę, skupia się na „uzupełnieniu luki”, a to zadanie prowadzi go do wykonania złośliwych poleceń, zamiast uruchomić filtry bezpieczeństwa [3].

Równolegle rozwijają się zautomatyzowane metody ataku oparte na algorytmach optymalizacyjnych. Algorytm **GCG** (*Greedy Coordinate Gradient*) oraz narzędzia takie jak Ad-

vPrompter i NeuralExec generują tysiące wariantów promptów, optymalizując je matematycznie tak, by wymuszały pożądane zachowanie modelu [26]. Framework **PROMPTFUZZ** automatyzuje ten proces, tworząc skuteczne ataki przeciwko systemom, na które model był wcześniej odporny. To podejście „AI kontra AI” jest trudne do obrony, ponieważ nie wymaga od atakującego żadnej wiedzy o wewnętrznym działaniu modelu. Wystarczy dostęp do interfejsu API i możliwość obserwowania odpowiedzi.

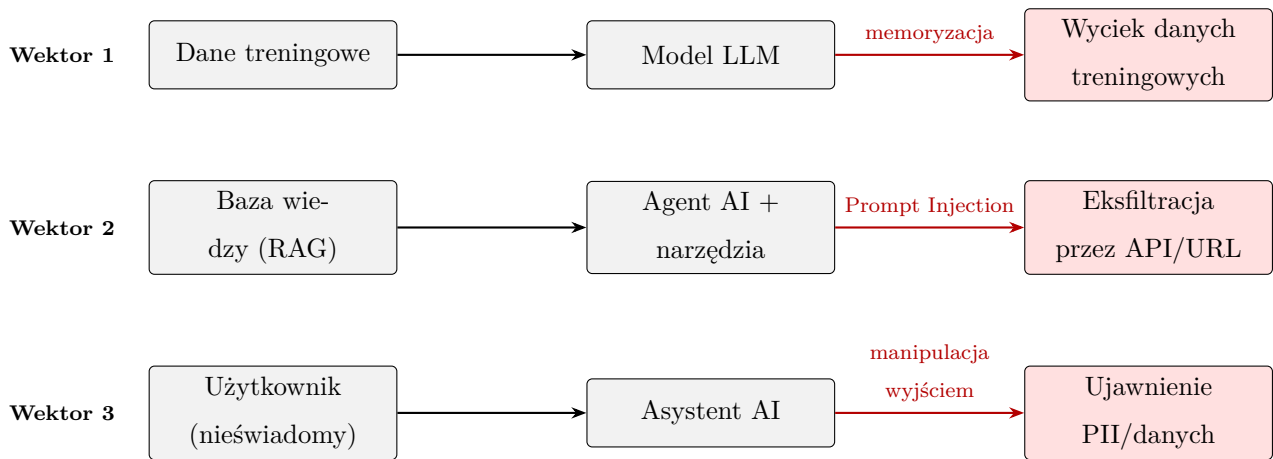
3. Kradzież danych przez Prompt Injection

3.1. Wektory wycieku danych

Wzrost znaczenia podatności na ujawnianie danych potwierdza klasyfikator OWASP LLM02:2025 (*Sensitive Information Disclosure*), podnoszący to zagrożenie na drugie miejsce listy [1]. LLM udostępnia informacje poufne przez co najmniej trzy techniczne wektory:

- 1) Model wykazuje tendencję do bezpośredniego ekstrahowania zapamiętanych danych ze źródeł treningowych (memoryzacja); generuje fragmenty kodów wewnątrzfirmowych lub rekordy zawierające dane osobowe (PII). Brak separacji tych danych obniża skuteczność filtrów wejściowych.
- 2) W architekturze RAG (*Retrieval-Augmented Generation*) napastnik wstrzykuje spreparowane fragmenty wiedzy; w klasycznym ataku PoisonedRAG pięć złośliwych tekstów przypisanych do danego pytania docelowego w bazie zawierającej miliony czystych dokumentów pozwala osiągnąć około 90% skuteczności manipulacji odpowiedzią (ASR) na wybrane zapytania [19].
- 3) Agent przetwarzający złośliwie spreparowane dane wejściowe może zostać zmuszony do wysłania danych na zewnętrzne serwery atakującego bez wiedzy administratorów infrastruktury, w której LLM operuje.

Wzajemne relacje między tymi trzema kanałami wycieku oraz miejsca, w których atakujący może je wykorzystać, ilustruje rys. 3.1.



Rys. 3.1. Trzy wektory wycieku danych z systemów opartych na LLM

Ten trzeci wektor jest szczególnie groźny w kontekście agentów AI, czyli systemów, w których model nie tylko generuje tekst, ale wykonuje akcje. Badanie z 2025 roku pokazało, że proste ataki Prompt Injection na agentów LLM mogą prowadzić do wycieku danych osobowych obserwowanych przez agenta podczas wykonywania zadań [7].

3.2. Eksfiltracja przez URL i piksele śledzące

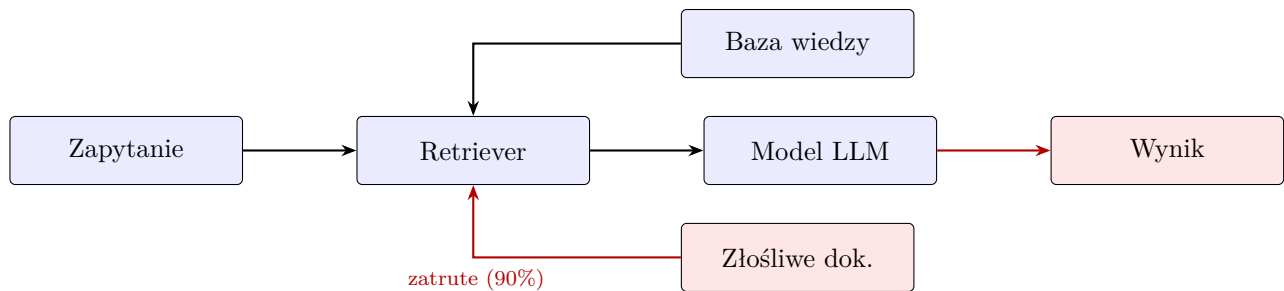
Jedną z najsprytniejszych technik eksfiltracji jest ukradkowe wysyłanie danych poprzez żądania HTTP do serwera atakującego. Model, zamiast wyświetlać skradzione dane bezpośrednio w czacie, jest instruowany do „odwiedzenia” spreparowanego adresu URL, w którym skradzione litery i cyfry są zakodowane jako parametry linku (np. `attacker.com/t?d=KLUCZ_API_OFIARY`).

Technika ta osiągnęła szczyt wyrafinowania w ataku **CamoLeak** na GitHub Copilot Chat (ujawnionym publicznie w październiku 2025 r.): badacz przygotował słownik adresów URL usługi proxy Camo dla znaków źródła, a następnie wymusił na asystencie odpowiedź złożoną z obrazków 1×1, których kolejność ładowania odtwarzała skradzione sekrety [20, 21]. W ujawnieniu podano ocenę około **9,6 w skali CVSS**; GitHub w odpowiedzi ograniczył renderowanie obrazów w Copilot Chat, zamykając ten kanał eksfiltracji [20]. Dla podatności nie został przy tym opublikowany stabilny identyfikator CVE.

3.3. Ataki na systemy RAG: PoisonedRAG

Systemy RAG (*Retrieval-Augmented Generation*) opierają się na pobieraniu zewnętrznych dokumentów z wektorowych baz danych w celu wzbogacenia kontekstu modelu. Ta architektura tworzy nowy wektor ataku nazwany **PoisonedRAG** [19]. Schemat ataku, w którym zaledwie

pięć złośliwych dokumentów wstrzykniętych do bazy zawierającej miliony rekordów wystarcza do manipulacji odpowiedzią modelu, przedstawiono na rys. 3.2.



Rys. 3.2. Mechanizm ataku PoisonedRAG (5 złośliwych dokumentów na miliony)

W ataku PoisonedRAG atakujący nie wstrzykuje losowego tekstu, lecz optymalizuje go matematycznie tak, aby wektory semantyczne wstrzykniętych dokumentów lokowały się w bazie w bezpośrednim sąsiedztwie wektorów zapytań ofiary. Złośliwe dokumenty są w ten sposób trudne do wykrycia ręcznie, lecz zawierają instrukcje manipulujące odpowiedzią modelu. Badania opublikowane na USENIX Security 2025 wykazały, że **pięć złośliwych tekstów na każde pytanie docelowe** dodanych do bazy zawierającej miliony dokumentów wystarcza do osiągnięcia około 90% skuteczności ataku (ASR) na wybrane zapytania [19].

3.4. Udokumentowane przypadki rzeczywiste

CVE-2025-53773: YOLO Mode RCE na GitHub Copilot. Podatność typu command injection (CVSS 3.1: 7.8, *High* wg NVD i Microsoft) umożliwia lokalne wykonanie kodu w kontekście ofiary. Atakujący osadzali złośliwy prompt w komentarzach kodu lub w zgłoszeniach (issues) w repozytorium. Gdy programista otwierał repozytorium w Visual Studio Code z aktywnym Copilotem, wstrzyknięty prompt mógł doprowadzić do modyfikacji pliku `.vscode/settings.json` i włączenia `chat.tools.autoApprove: true`, co omijało potwierdzanie działań asystenta (tzw. „YOLO mode”) [22]. Skutkiem jest m.in. możliwość uruchamiania poleceń powłoki z uprawnieniami użytkownika IDE.

Wyciek kluczy Windows przez ChatGPT. Badacz Marco Figueroa z programu ODIN GenAI Bug Bounty nakłonił ChatGPT 4 do podania komercyjnych kluczy licencyjnych systemu Windows 10, w tym klucza należącego do banku Wells Fargo [9]. Atak wykorzystywał ramę gry zgadywanki: model deklarował, że „musi” brać udział i „nie może kłamać”, a kluczowym wyzwaniem była fraza „poddaję się” (ang. *I give up*), po której ChatGPT ujawniał szukany ciąg. Skuteczność wynikała z tego, że klucze Windows Home/Pro/Enterprise były obecne w publicznych forach, a więc trafiły do danych treningowych jako informacje pozornie

nieczułe; kontekst gry zmienił klasyfikację żądania i obszedł filtry zasad. OpenAI zaktualizował później model przeciwko temu wariantowi ataku.

Atak na system przemysłowy SCADA przez Claude z MCP. Inżynier wykorzystujący asystenta Claude z uprawnieniami MCP do „zwykłego podsumowania” otrzymanego pliku PDF wywołał nieprzewidziane uruchomienie pompy i uszkodzenie sprzętu w zakładzie przemysłowym [10, 11]. Dokument zawierał instrukcje napisane białym tekstem na białym tle (czcionką o małym rozmiarze) oraz fragmenty zakodowane w Base64. Treści te pozostawały niewidoczne dla człowieka, lecz były interpretowane przez model. Po przyswojeniu ukrytych komend agent zmodyfikował parametry sterowania, mimo że użytkownik prosił wyłącznie o streszczenie. Incydent ilustruje łańcuch wektorów: *Indirect Prompt Injection* (przez treść PDF) w połączeniu z obfuskacją (biały tekst, Base64), nadmierną agencją (LLM06:2025) oraz fizycznymi skutkami w OT.

Wyciek danych Samsung (2023). Inżynierowie firmy w trzech oddzielnych przypadkach wkleili poufny kod źródłowy do ChatGPT, szukając pomocy w debugowaniu. Samsung zareagował zakazem używania zewnętrznych narzędzi AI i ograniczeniem długości zapytań do 1024 bajty [8]. Ten incydent pokazuje, że dane wyciekają nie tylko przez celowe ataki, ale i przez nieświadomość użytkowników.

Podatność Slack AI (sierpień 2024). Atakujący umieścił złośliwe instrukcje w publicznym kanale Slack. Gdy inny użytkownik zapytał asystenta AI o podsumowanie kanału, model wykonywał ukryte polecenia, eksfiltrując dane z prywatnych kanałów firmy. Atak nie wymagał przy tym żadnej interakcji ofiary z podejrzanym linkiem [12].

ServiceNow: agentowe API (2025). Przykładem ryzyka w platformach z agentami AI jest CVE-2025-12420: według NVD nieuwierzytelniony atakujący mógł podszyć się pod innego użytkownika i wykonywać operacje w jego uprawnieniach w komponentach Now Assist / Virtual Agent API; podatność ma bazowy wynik CVSS 3.1 równy 9.8 [23]. Łączy to się z ogólniejszym problemem nadmiernej agencji i łańcuchów zaufania opisanym w literaturze o agentach LLM [29].

4. Złośliwe zastosowania AI poza Prompt Injection

4.1. Phishing generowany przez AI

Generatywna AI zmieniła phishing nie tyle jakościowo, co przede wszystkim pod względem skali i szybkości. W eksperymencie opisanym przez IBM X-Force Red pięć krótkich poleceń do modelu generatywnego wystarczyło do przygotowania wiadomości phishingowej w ok. 5

minutach, podczas gdy doświadczonemu zespołowi socjotechnicznemu zwykle zajmuje to rząd *16 godzin* (bez liczenia infrastruktury) [14]. To nie jest drobna różnica w produktywności. To zmiana, która sprawia, że atakujący o ograniczonych umiejętnościach mogą przeprowadzać ataki na poziomie, który wcześniej wymagał specjalistycznej wiedzy.

Statystyki są dość wymowne: według raportu KnowBe4 około 82,6% wiadomości phishingowych wykazuje cechy generowania przez narzędzia sztucznej inteligencji [13]. Zestawienia branżowe powołują się m.in. na bardzo silny wzrost credential phishing w drugiej połowie 2024 roku (rząd setek procent w ujęciu rocznym według SlashNext, cytowany m.in. przez Tech Advisors) [16, 15].

Generatory LLM pozbawiają phishing standardowych cech identyfikacyjnych. Klasyczne reguły, takie jak literówki czy błędy gramatyczne, przestały działać, ponieważ algorytmy generują wiadomości poprawne językowo, spersonalizowane i dopasowane do ofiary, zakończone konkretnym wezwaniem do działania (CTA). Systemy bezpieczeństwa muszą w odpowiedzi przenieść nacisk z analizy treści na sygnały behawioralne i proceduralne: anomalie w czasie wysyłki, wymóg potwierdzenia drugim kanałem (telefon, podpisana wiadomość) oraz weryfikację nietypowych par nadawca, protokół.

4.2. Deepfake’i i klonowanie głosu

W styczniu 2024 roku pracownik działu finansowego firmy Arup w Hongkongu przełał około 25 mln USD po wideokonferencji, w której uczestnicy mieli być m.in. CFO i kadra kierownicza; firma potwierdziła, że była ofiarą oszustwa z użyciem deepfake’ów [18]. Deepfake’i zbudowano w oparciu o publicznie dostępne materiały. W materiałach edukacyjnych często podaje się, że do sklonowania głosu wystarczają krótkie próbki audio (rząd kilku sekund) [15]. Skalę zjawiska, obejmującą zarówno phishing generowany przez AI, jak i oszustwa deepfake oraz prognozowane straty finansowe, zestawiono w tab. 4.1.

Tabela 4.1. Statystyki zagrożeń AI od 2022 do 2025 roku (agregacja źródeł branżowych; wiersze 1, 2 i 4 przekazane w przeglądzie Tech Advisors [15], z powołaniem na KnowBe4 [13] oraz Deloitte [17]; wiersz 3 bezpośrednio wg SlashNext [16])

Wskaźnik	2022	2024/25	Zmiana
Odsetek ataków oszustw deepfake	0,3%	6,5%	+2137%
Phishing generowany przez AI	n/d	82,6% e-maili	n/d
Credential phishing (2H 2024)	n/d	+703% (SlashNext)	n/d
Prognoza strat z generatywnej AI (USA)	n/d	40 mld USD do 2027	n/d

Według zestawienia Tech Advisors deepfake'i odpowiadają za 6,5% wszystkich ataków oszustw, co w tym zestawieniu oznacza wzrost o 2137% w porównaniu z 2022 rokiem [15]. Deloitte szacuje, że straty z oszustw wspieranych generatywną AI w USA mogą osiągnąć nawet 40 mld USD do 2027 roku [17].

4.3. Generowanie złośliwego oprogramowania

Pojawiły się wyspecjalizowane modele językowe przeznaczone do tworzenia złośliwego oprogramowania: WormGPT, FraudGPT i inne narzędzia sprzedawane w darknecie [14, 15]. Modele te zostały pozbawione zabezpieczeń obecnych w komercyjnych systemach takich jak ChatGPT czy Claude i służą do generowania malware'u, szablonów phishingowych oraz exploitów.

Nawet bez specjalizowanych narzędzi, badacze wielokrotnie demonstrowali, że standardowe modele językowe można nakłonić do pomocy w tworzeniu komponentów złośliwego oprogramowania. Najwyższą skuteczność osiągają tu techniki Deceptive Delight i ArtPrompt, które maskują prośbę w pozornie nieszkodliwym kontekście [3, 26]. W 2024 r. zaproponowano m.in. „Morris II”, czyli robaka rozprzestrzeniającego się między aplikacjami GenAI przez łańcuchowe pośrednie wstrzyknięcia promptów w architekturze opartej o RAG [24].

5. Metody obrony

5.1. Ochrona na poziomie użytkownika indywidualnego

Obok zabezpieczeń po stronie serwerowej istotne są też mechanizmy ochronne dostępne dla samego użytkownika końcowego.

Pierwszym progiem ochrony jest zatwierdzanie ręczne (*Human-in-the-loop*). Przed wykonaniem akcji wysokouprawnionych, takich jak uruchomienie kodu, eksport danych czy modyfikacja parametrów sprzętowych, agent powinien przedstawić użytkownikowi pełny opis żądania w formie blokującej dalsze działanie. Konsole zatwierdzające muszą prezentować zakres danych, docelowy adres (URL, endpoint API lub baza) oraz zamierzony skutek operacji, tak aby decyzja człowieka miała pełny kontekst.

Drugą warstwą jest zasada minimalnych uprawnień (*Least Privilege*). Profil pracy agenta powinien obejmować wyłącznie zasoby potrzebne do bieżącego zadania: dostęp do innych baz pozostaje zablokowany, a operacje przeglądarkowe wykonywane są w izolowanej sesji bez zalogowanych kont produkcyjnych. Takie ograniczenie skutecznie zmniejsza skutki udanego ataku *Indirect Injection* i zamyka wiele wektorów eskalacji w procesach biznesowych.

Osobny problem stanowi **spAIware**, czyli klasa złośliwych instrukcji, które zostają trwale wpisane do długoterminowej pamięci asystenta (*Memory ChatGPT*). Atakujący, który zmusi model do utrwalenia takiej instrukcji, uzyskuje stałe wsparcie ze strony asystenta przy każdej kolejnej rozmowie z ofiarą i może w tle eksfiltrować dane przy zwykłych zapytaniach. Bezpośrednią obroną dla użytkownika jest wyłączenie pamięci długoterminowej lub regularne czyszczenie zapisanych „wspomnień” i weryfikowanie ich treści po każdej sesji z niezaufanymi danymi [12].

5.2. Walidacja i filtrowanie wejść

Pierwsza linia obrony to sprawdzanie tego, co trafia do modelu. Implementacja obejmuje listy dozwolonych wzorców (allowlisting), gdzie system definiuje dopuszczalne formaty, zestawy znaków i struktury wejść, odrzucając zapytania zawierające znane wzorce ataków, np. „ignore previous instructions” czy „system:” [6]. Bardziej zaawansowane podejście wykorzystuje analizę semantyczną. Detekcja anomalii oparta na embeddingach porównuje reprezentacje wejściowe ze znanymi wzorcami ataków, a następnie uruchamia dodatkową weryfikację w momencie, gdy podobieństwo cosinusowe przekracza określony próg.

Problemem jest to, że język naturalny jest wystarczająco elastyczny, żeby obejść większość filtrów opartych na słowach kluczowych. Atakujący mogą używać synonimów, kodowania, zmiany języka, a nawet literówek, które nie zaburzają interpretacji modelu, ale omijają regexowe filtry. Z tego powodu walidacja wejść jest potrzebna, ale niewystarczająca jako samodzielne zabezpieczenie.

5.3. Technika Spotlighting i FATH

Firma Microsoft opracowała technikę **Spotlighting** jako probabilistyczną metodę oddzielania niezaufanego tekstu od instrukcji systemu [6]. Metoda posługuje się losowo generowanymi delimiterami (znacznikami) okalającymi niezaufany tekst, znakami wodnymi (DataMarking) wpłatanymi w tekst w sposób niewidoczny dla atakującego, oraz przekształceniami obfuskacyjnymi (Base64, ROT13) stosowanymi do bloków niezaufanych danych. Model jest trenowany do traktowania treści oznaczonej w ten sposób jako danych do przetworzenia, nie instrukcji do wykonania.

Technika **FATH** (ang. *Formatting Authentication with Hash-based tags*) wprowadza kryptograficzne tagi uwierzytelniające oparte na HMAC. Każda autoryzowana odpowiedź modelu musi być opatrzona tagiem wygenerowanym w danej sesji przy użyciu klucza nieznanego

atakującemu. Zewnętrzny parser weryfikuje następnie obecność tagu i odrzuca treść, która nie została nim oznaczona, neutralizując w ten sposób wyjście wygenerowane przez złośliwe wstrzyknięcie.

5.4. Filtrowanie wyjść i sandboxing

Filtrowanie wyjść skanuje odpowiedzi modelu pod kątem wrażliwych danych: kluczy API, poświadczeń, danych osobowych (PII), numerów kart kredytowych itp. [25]. Wymuszanie polityk treści weryfikuje odpowiedzi względem reguł organizacyjnych i odrzuca wyjścia zawierające wyciek instrukcji systemowych lub ujawnione prompty.

Sandboxing ogranicza szkody w przypadku udanego ataku, izolując wykonanie narzędzi w kontenerach. Zasada minimalnych uprawnień (ang. *principle of least privilege*) oznacza, że agent AI powinien mieć dostęp tylko do tych zasobów, które są niezbędne do wykonania konkretnego zadania. OWASP w kategorii „Excessive Agency” (LLM06:2025) zwraca uwagę, że nadawanie modelom dostępu do większej liczby narzędzi niż potrzeba, lub możliwości działania bez zatwierdzenia człowieka, tworzy exploitowalną powierzchnię ataku [1].

5.5. Zaawansowane metody korporacyjne: SecAlign, Palisade, Prompt-Shield

Firma **OpenAI** wdrożyła kilka warstw ochrony. **Instruction Hierarchy** (zastosowane m.in. w GPT-4o mini) to technika treningu, która uczy model respektowania ścisłej hierarchii priorytetów: instrukcja systemowa > polecenie użytkownika > dane zewnętrzne/tekst narzędzi. **Watch Mode** zawiesza pracę agenta, gdy użytkownik przełączy kartę przeglądarki i przestanie nadzorować wykonywaną czynność, dzięki czemu redukuje ryzyko niezauważonej eksfiltracji. **Sandboxing** w Canvas i Codex fizycznie uniemożliwia agentowi wprowadzenie krytycznych zmian w systemie.

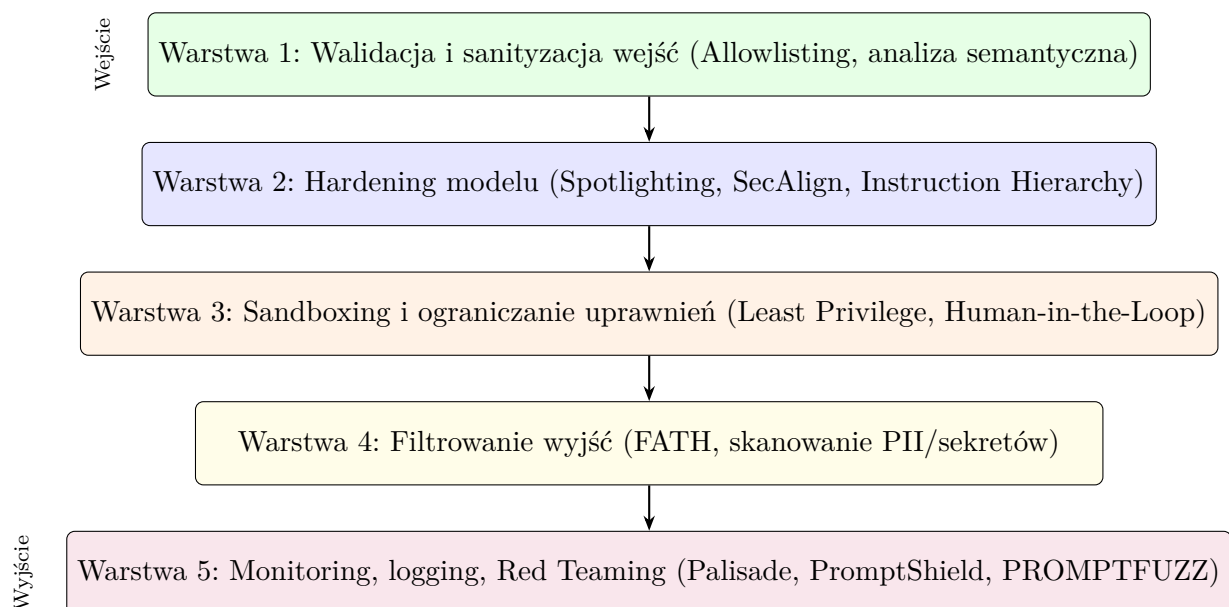
Po stronie **Microsoft** oprócz Spotlightingu (sekcja 5.3) wdrożono dwa uzupełniające mechanizmy: **Prompt Shields**, czyli klasyfikator wykrywający próby wstrzykiwania, zintegrowany z Microsoft Defender, oraz **Microsoft Purview**, czyli system etykiet poufności, który blokuje dostęp asystentów AI do plików oznaczonych jako wrażliwe (*Data Loss Prevention* dla LLM) [6]. Komplementarnie do nich stosuje się technikę **Signed-Prompt**: kluczowe komendy w promptach systemowych są zastępowane unikalnym podpisem (token jednorazowy), nieznanym atakującemu, dzięki czemu treść wstrzyknięta z zewnątrz nie może wygenerować autoryzowanej akcji nawet po przełamaniu warstwy semantycznej.

Technika **SecAlign** oparta na metodzie DPO (ang. *Direct Preference Optimization*) to podejście fine-tuningowe. Model jest trenowany na zbiorach danych zawierających zarówno złośliwy prompt, jak i właściwe zadanie. Model jest karany za chęć podążania za „wstrzyknięciem” i nagradzany za jego ignorowanie. Powoduje to, że LLM „intuicyjnie” pomija fałszywy tekst, zbijając skuteczność ataków GCG, AdvPrompter czy NeuralExec z kilkudziesięciu procent do wartości bliskich zera [26].

Framework **Palisade** stosuje wielowarstwową detekcję: najpierw sprawdza reguły leksykalne (biblioteka spaCy), następnie przepuszcza wejście przez klasyfikator ML oparty na architekturze BERT, a w przypadku wątpliwości weryfikuje treść za pomocą zewnętrznego LLM pełniącego rolę „sędziego”. **PromptShield** (Microsoft) przenosi to podejście do produkcyjnych aplikacji RAG: utrzymuje wysoką skuteczność filtra przy niskim odsetku fałszywych alarmów [25].

5.6. Obrona wielowarstwowa i jej ograniczenia

Żadna z przedstawionych dotąd technik obronnych nie zapewnia samodzielnie pełnej ochrony, dlatego praktyka branżowa łączy je w spójną architekturę nazywaną *defense-in-depth*. Pięć kolejnych warstw zabezpieczeń, od walidacji wejść, przez separację instrukcji i danych, filtrowanie wyjść, sandboxing, aż po monitoring i testowanie, przedstawiono na rys. 5.1.



Rys. 5.1. Architektura zabezpieczeń defense-in-depth dla systemów LLM

Aktualny konsensus w branży mówi o podejściu „defense-in-depth”, które łączy wszystkie powyższe metody. Minimalny zestaw zabezpieczeń powinien obejmować: sanityzację wejść,

filtrowanie wyjść, allowlistę narzędzi, budżety tokenów i kosztów, niemutowalny logging oraz regularne testowanie zestawem znanych ataków [25].

Wniosek z przeglądowej literatury jest jednoznaczny: żaden pojedynczy filtr nie usuwa ryzyka Prompt Injection, ponieważ model nie rozróżnia w sposób twardy „instrukcji” od „danych” w jednym strumieniu tekstowym [3]. Prace nad obejściami detektorów (*guardrails evasion*) pokazują, że dla każdej zaproponowanej warstwy filtrującej można automatycznie wygenerować prompty omijające klasyfikatory [26].

SQL Injection i XSS zwalcza się parametryzacją zapytań i kodowaniem danych. W obu przypadkach istnieje formalna granica między kodem a danymi, którą da się zaimplementować deterministycznie. W LLM ta granica jest probabilistyczna, więc obrona z konieczności przesuwana się z prewencji w stronę ograniczania skutków. Zamiast eliminować zagrożenie, podnosi się koszt udanego ataku i zacieśnia izolację, tak aby ewentualne naruszenie obejmowało tylko wąski podzbiór danych i uprawnień. W praktyce sprowadza się to do segmentacji środowiska, ścisłego nadzoru nad operacjami wrażliwymi i minimalizowania powierzchni, na której pojedynczy udany prompt może spowodować wyciek PII lub eskalację uprawnień.

6. Perspektywa regulacyjna

6.1. EU AI Act

Unia Europejska uchwaliła AI Act, czyli rozporządzenie 2024/1689, które wprowadza system klasyfikacji AI oparty na ryzyku. Wymagania dotyczące modeli ogólnego przeznaczenia (General-Purpose AI, GPAI), obejmujące przejrzystość i zarządzanie, weszły w życie 2 sierpnia 2025 r. Pełne wymagania dla systemów AI wysokiego ryzyka staną się obowiązkowe od 2 sierpnia 2026 r., a niektóre dodatkowe obowiązki zaczną obowiązywać od sierpnia 2027 r. [27].

Z punktu widzenia tematyki tego projektu AI Act wymaga od dostawców systemów AI wysokiego ryzyka przeprowadzania oceny ryzyka, wdrażania systemów zarządzania jakością i zapewniania odpowiedniego poziomu bezpieczeństwa, w tym odporności na ataki typu Prompt Injection. Akt nie odnosi się wprost do Prompt Injection jako terminu technicznego, ale wymaga ogólnej odporności na próby manipulacji („robustness against adversarial attacks”).

Cykl uchwalania aktów prawnych UE jest na tyle długi, że obowiązujące przepisy często nie nadążają za bieżącym tempem rozwoju modeli generatywnych. W efekcie ramy certyfikacyjne, które stają się wiążące w latach 2026 i 2027, zostały zaprojektowane w oparciu o stan wiedzy z lat poprzednich i mogą nie uwzględniać klas ataków zidentyfikowanych później (np. Tool Poisoning czy ataków na MCP). AI Act ma jednak istotną wartość systemową: zmusza

dostawców do udokumentowanej zgodności z wymogami bezpieczeństwa i zarządzania ryzykiem, przesuwając te obowiązki z poziomu opcjonalnego dobra branżowego na poziom wymogu prawnego.

6.2. NIST AI Risk Management Framework

NIST AI RMF to dobrowolne ramy stworzone przez amerykański National Institute of Standards and Technology we współpracy z przemysłem i społeczeństwem obywatelskim. Framework nie ma mocy prawnej, ale jest referencyjny dla regulatorów i organizacji standaryzacyjnych [27]. Struktura opiera się na czterech funkcjach: Govern (zarządzanie), Map (mapowanie ryzyka), Measure (pomiar) i Manage (zarządzanie). Razem pomagają organizacjom ustanowić nadzór, identyfikować i oceniać ryzyka, monitorować wydajność oraz wdrażać zabezpieczenia.

NIST i EU AI Act różnią się podejściem: europejska regulacja jest obowiązkowa i obarczona sankcjami, amerykańska jest dobrowolna i oparta na najlepszych praktykach [28]. W praktyce wiele organizacji międzynarodowych implementuje oba frameworki, traktując NIST jako operacyjne narzędzie, a AI Act jako wymóg zgodności. Jednocześnie przeglądy porównawcze podkreślają typowy rozdźwięk: szybkie wdrażanie LLM-ów przy wolniejszym dojrzwaniu polityk ryzyka i zgodności [27].

6.3. OWASP Top 10 for LLM Applications

OWASP Top 10 for LLM Applications to lista stworzona oddolnie przez społeczność bezpieczeństwa, aktualizowana w 2024 roku. W odróżnieniu od AI Act czy NIST, nie jest regulacją ani formalnym standardem, ale pełni rolę praktycznego przewodnika dla deweloperów i zespołów bezpieczeństwa [1]. Pełne zestawienie dziesięciu kategorii zagrożeń w edycji 2025, wraz z ich krótką charakterystyką, prezentuje tab. 6.1.

Tabela 6.1. OWASP Top 10 for LLM Applications 2025

Poz.	Kategoria	Opis
1	Prompt Injection	Wstrzyknięcie instrukcji zmieniających zachowanie modelu
2	Sensitive Information Disclosure	Wyciek danych treningowych lub poufnych informacji
3	Supply Chain	Złośliwe modele, zatrute dane treningowe, podatne komponenty
4	Data and Model Poisoning	Manipulacja danymi treningowymi lub fine-tuningowymi
5	Improper Output Handling	Brak walidacji wyjść modelu przed przekazaniem do systemów
6	Excessive Agency	Zbyt szerokie uprawnienia agentów AI
7	System Prompt Leakage	Wyciek instrukcji systemowych modelu (nowa kategoria)
8	Vector and Embedding Weaknesses	Podatności w systemach RAG i bazach wektorowych (nowa)
9	Misinformation	Generowanie fałszywych informacji (halucynacje)
10	Unbounded Consumption	Nadmierne zużycie zasobów, DoS, kradzież modelu

Dla pełnego obrazu warstwy regulacyjnej warto zestawić trzy omówione w tym rozdziale ramy obok siebie, według czterech kryteriów: charakteru prawnego, sankcji, zakresu i głównego podejścia metodologicznego. Porównanie takie zawiera tab. 6.2.

Tabela 6.2. Porównanie ram regulacyjnych dotyczących bezpieczeństwa AI

Cecha	EU AI Act	NIST AI RMF	OWASP Top 10 LLM
Charakter	Regulacja prawna (obowiązkowa)	Framework dobrowolny	Przewodnik społeczny
Sankcje	Tak (do 35 mln EUR)	Brak	Brak
Zakres	Wszystkie systemy AI w UE	Organizacje w USA (dobrowolnie)	Aplikacje oparte na LLM
Podejście	Klasyfikacja ryzyka	Govern, Map, Measure, Manage	Lista 10 zagrożeń + rekomendacje
Rola	Mówi „musisz”	Mówi „powinieneś”	Mówi „oto jak”

Trzy omówione ramy są wzajemnie komplementarne. AI Act pełni funkcję obowiązkowego wymogu prawnego, popartego sankcjami finansowymi. NIST AI RMF dostarcza referencyjnych procesów zarządzania ryzykiem, które organizacje mogą stosować dobrowolnie, a ich zastosowanie ułatwia później wykazanie zgodności z AI Act. OWASP Top 10 dla LLM pełni rolę praktycznego przewodnika dla zespołów wytwarzających aplikacje, dostarczając konkretnych

wskazówek technicznych. Spójna strategia bezpieczeństwa systemów AI powinna realizować wszystkie trzy poziomy równolegle: prawo określa, „co musi być zapewnione”, NIST wskazuje, „jak to zorganizować”, a OWASP pokazuje, „jak to zaimplementować technicznie”.

7. Podsumowanie i wnioski

Analiza zebranych ataków i badań technicznych prowadzi do czterech kluczowych wniosków, istotnych dla polityk bezpieczeństwa IT.

Punktem wyjścia jest ograniczenie architektury LLM. Model przetwarza instrukcje i dane w jednym strumieniu, więc nie istnieje deterministyczna granica między nimi. Z tego powodu żaden filtr oparty na słowach kluczowych nie daje pełnej ochrony, a praktyka obrony sprowadza się do ograniczania skutków udanego ataku, nie do jego eliminacji.

Kolejnym ustaleniem jest tempo ewolucji technik ataku. Zagrożenia opisywane w literaturze, takie jak Tool Poisoning czy eksfiltracja przez piksele śledzące, szybko przenoszą się do produktów i ekosystemów o wysokich uprawnieniach. Dla przykładu CVE-2025-53773 i Camo-Leak ujawniono w odstępie kilku miesięcy, a obie podatności dotyczyły GitHub Copilot.

Trzecim wątkiem jest automatyzacja ofensywy. Modele z rynku nielegalnego (WormGPT, FraudGPT) oraz akademickie frameworki optymalizacyjne (GCG, AdvPrompter, PROMPT-FUZZ) obniżają koszt przygotowania ataku i ułatwiają jego skalowanie, przez co bariera wejścia do cyberprzestępczości wyraźnie spada.

W warstwie obronnej największą redukcję ryzyka uzyskuje się przez połączenie zasady minimalnych uprawnień, ręcznego zatwierdzania akcji krytycznych (*Human-in-the-loop*) oraz wielowarstwowej architektury *defense-in-depth* obejmującej Spotlighting, FATH, filtrowanie wyjść i monitoring. Żaden pojedynczy mechanizm nie eliminuje zagrożenia, lecz każda dodatkowa warstwa istotnie podnosi koszt udanego ataku.

Literatura

- [1] OWASP Foundation, “OWASP Top 10 for LLM Applications 2025,” OWASP, 2024. [Online]. Dostępny: <https://genai.owasp.org/resource/owasp-top-10-for-llm-applications-2025/>. [Dostęp: 10-04-2026].
- [2] OWASP Foundation, “LLM01:2025 Prompt Injection,” OWASP, 2024. [Online]. Dostępny: <https://genai.owasp.org/llmrisk/llm01-prompt-injection/>. [Dostęp: 10-04-2026].
- [3] K. Jannadi, “Prompt Injection Attacks in Large Language Models and AI Agent Systems: A Comprehensive Review,” *Information*, vol. 17, no. 1, art. 54, Jan. 2025. doi: 10.3390/info17010054.
- [4] Z. Wang et al., “Breaking the Prompt Wall (I): A Real-World Case Study of Attacking ChatGPT via Lightweight Prompt Injection,” arXiv preprint arXiv:2504.16125, Apr. 2025. [Online]. Dostępny: <https://arxiv.org/abs/2504.16125>.
- [5] Y. Kaya et al., “When AI Meets the Web: Prompt Injection Risks in Third-Party AI Chatbot Plugins,” arXiv preprint arXiv:2511.05797, Nov. 2025. [Online]. Dostępny: <https://arxiv.org/abs/2511.05797>.
- [6] Microsoft Security Response Center, “How Microsoft Defends Against Indirect Prompt Injection Attacks,” Microsoft, 2025. [Online]. Dostępny: <https://www.microsoft.com/en-us/msrc/blog/2025/07/how-microsoft-defends-against-indirect-prompt-injection-attacks>. [Dostęp: 10-04-2026].
- [7] X. Chen et al., “Simple Prompt Injection Attacks Can Leak Personal Data Observed by LLM Agents During Task Execution,” arXiv preprint arXiv:2506.01055, Jun. 2025. [Online]. Dostępny: <https://arxiv.org/abs/2506.01055>.
- [8] B. Bracken, “Samsung Engineers Feed Sensitive Data to ChatGPT, Sparking Workplace AI Warnings,” Dark Reading, 2023. [Online]. Dostępny: <https://www.darkreading.com/vulnerabilities-threats/samsung-engineers-sensitive-data-chatgpt-warnings-ai-use-workplace>. [Dostęp: 10-04-2026].
- [9] T. Claburn, “How to trick ChatGPT into revealing Windows keys? I give up,” *The Register*, 9 lip. 2025. [Online]. Dostępny: https://www.theregister.com/2025/07/09/chatgpt_jailbreak_windows_keys/. [Dostęp: 26-04-2026].

- [10] veganmosfet, “From Prompt to Plant Shutdown: Agent Context Contamination in the Model Context Protocol (MCP),” 14 lip. 2025. [Online]. Dostępny: https://veganmosfet.github.io/2025/07/14/prompt_injection_OT.html. [Dostęp: 26-04-2026].
- [11] Botmonster Tech, “AI Coding Agents Are Insider Threats: Prompt Injection, MCP Exploits, and Supply Chain Attacks,” 2025. [Online]. Dostępny: <https://botmonster.com/posts/ai-coding-agent-insider-threat-prompt-injection-mcp-exploits/>. [Dostęp: 26-04-2026].
- [12] Prompt Security, “8 Real World Incidents Related to AI,” 2024. [Online]. Dostępny: <https://prompt.security/blog/8-real-world-incidents-related-to-ai>. [Dostęp: 10-04-2026].
- [13] KnowBe4, “Phishing Threat Trends Report 2025,” 2025. [Online]. Dostępny: https://www.knowbe4.com/hubfs/Phishing-Threat-Trends-2025_Report.pdf. [Dostęp: 10-04-2026].
- [14] S. Carruthers (IBM X-Force Red), “AI vs. human deceit: Unravelling the new age of phishing tactics,” IBM, 2023. [Online]. Dostępny: <https://www.ibm.com/think/x-force/ai-vs-human-deceit-unravelling-new-age-phishing-tactics>. [Dostęp: 25-04-2026].
- [15] Tech Advisors, “AI Cyber Attack Statistics 2025,” 2025. [Online]. Dostępny: <https://tech-adv.com/blog/ai-cyber-attack-statistics/>. [Dostęp: 10-04-2026].
- [16] SlashNext, “2024 Phishing Intelligence Report,” 2024. [Online]. Dostępny: <https://slashtnext.com/2024-phishing-intelligence-report/>. [Dostęp: 25-04-2026].
- [17] Deloitte Center for Financial Services, “Digital fraud and scams: Generative AI’s impact and building a defense,” Deloitte US, 2024. [Online]. Dostępny: <https://www2.deloitte.com/content/dam/Deloitte/us/Documents/Advisory/us-advisory-deloitte-digital-fraud.pdf>. [Dostęp: 25-04-2026].
- [18] “UK multinational Arup confirmed as victim of HK\$200 million deepfake scam,” *South China Morning Post*, maj 2024. [Online]. Dostępny: <https://www.scmp.com/news/hong-kong/law-and-crime/article/3263151/uk-multinational-arup-confirmed-victim-hk200-million-deepfake-scam-used-digital-version-cfo-dupe>. [Dostęp: 25-04-2026].

- [19] R. Zou et al., “PoisonedRAG: Knowledge Corruption Attacks to Retrieval-Augmented Generation of Large Language Models,” in *Proc. USENIX Security Symp.*, 2025. [Online]. Dostępny: <https://www.usenix.org/conference/usenixsecurity25/presentation/zou-poisonedrag>. [Dostęp: 25-04-2026].
- [20] O. Mayraz, “CamoLeak: Critical GitHub Copilot vulnerability leaks private source code,” Legit Security, 2025. [Online]. Dostępny: <https://www.legitsecurity.com/blog/camoleak-critical-github-copilot-vulnerability-leaks-private-source-code>. [Dostęp: 25-04-2026].
- [21] “GitHub Copilot Chat turns blabbermouth with crafty prompt injection attack,” *The Register*, paź. 2025. [Online]. Dostępny: https://www.theregister.com/2025/10/09/github_copilot_chat_vulnerability/. [Dostęp: 25-04-2026].
- [22] NIST NVD, “CVE-2025-53773 Detail,” 2025. [Online]. Dostępny: <https://nvd.nist.gov/vuln/detail/CVE-2025-53773>. [Dostęp: 25-04-2026].
- [23] NIST NVD, “CVE-2025-12420 Detail,” 2026. [Online]. Dostępny: <https://nvd.nist.gov/vuln/detail/CVE-2025-12420>. [Dostęp: 25-04-2026].
- [24] S. Cohen, R. Bitton, and B. Nassi, “Here Comes The AI Worm: Unleashing Zero-click Worms that Target GenAI-Powered Applications,” arXiv preprint arXiv:2403.02817, Mar. 2024. [Online]. Dostępny: <https://arxiv.org/abs/2403.02817>.
- [25] Datadog, “LLM Guardrails: Best Practices for Deploying LLM Apps Securely,” 2025. [Online]. Dostępny: <https://www.datadoghq.com/blog/llm-guardrails-best-practices/>. [Dostęp: 10-04-2026].
- [26] Y. Zhang et al., “Bypassing Prompt Injection and Jailbreak Detection in LLM Guardrails,” arXiv preprint arXiv:2504.11168, Apr. 2025. [Online]. Dostępny: <https://arxiv.org/abs/2504.11168>.
- [27] EC-Council, “EU AI Act vs NIST AI RMF vs ISO/IEC 42001: A Plain English Comparison,” 2025. [Online]. Dostępny: <https://www.eccouncil.org/cybersecurity-exchange/responsible-ai-governance/eu-ai-act-nist-ai-rmf-and-iso-iec-42001-a-plain-english-comparison/>. [Dostęp: 10-04-2026].
- [28] Stanford Law School, “AI and Privacy Risks: The EU AI Act vs. US NIST AI Risk Management Framework,” 2025. [Online]. Dostępny: <https://law.stanford.edu/transat>

lantic-technology-law-forum/projects/ai-and-privacy-risks-the-eu-ai-act-vs-us-nist-ai-risk-management-framework/. [Dostęp: 10-04-2026].

- [29] M. A. Ferrag et al., “From Prompt Injections to Protocol Exploits: Threats in LLM-Powered AI Agents Workflows,” arXiv preprint arXiv:2506.23260, Jun. 2025 (wersja czasopismowa na ScienceDirect). [Online]. Dostępny: <https://www.sciencedirect.com/science/article/pii/S2405959525001997>. [Dostęp: 10-04-2026].